Programa:	SWiSHpix
Protección:	Empakado con PEBundle 2.0x - 2.4x + TimeTrial
Objetivos:	Desempakar y evitar las limitaciones
URL descarga:	http://www.swishzone.com:8080/SetupSwishpix.exe
Dificultad:	No es difícil
Herramientas:	OllyDbg 1.10 + PEiD 0.93 + Import Reconstructor 1.6 + SnagIt (para imagenes)
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinos] Fecha: 23-10-2005

Hola a todos una vez más..!!

hoy trataremos un programa que he bajado hoy mismo gracias a _JIUS_ que envió a la lista unos links para descargar una serie de programas de este tipo todos de la compañía Swish, lo cierto es que además envió un link para descargar los parches pero yo soy amante del cracking como tú que estás leyendo esto y no me gustan los cracks regalados, además escuché que iban con password y no sé si lo enviaron a la lista, de cualquier modo nosotros vamos a intentar obtener un ejecutable libre de PEBundle y totalmente funcional, al menos eso intentaremos enseguida ;-)

Lo primero de todo y como siempre hacemos es abrir el exe en PEiD/RDG Packer versión 0.5.8 que van ya y veremos si está empakado/encriptado:

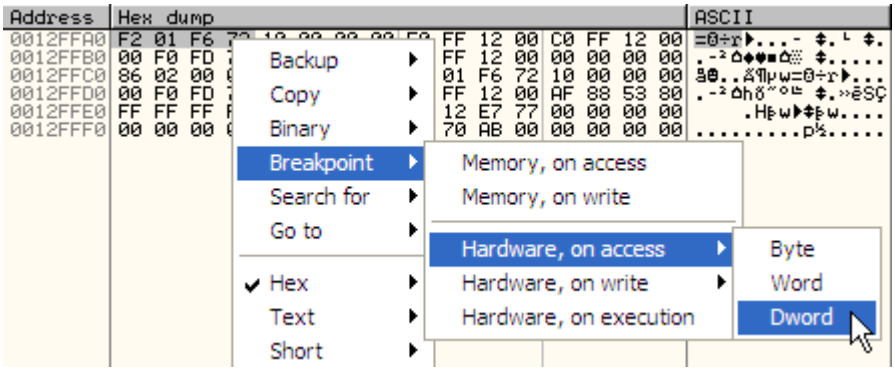
PEBundle 2.0x - 2.4x-> Jeremy Collake

En este caso sí está empakado con PEBundle, este protector no creo que nos dé muchos problemas porque por lo que he leído sobre él no redirecciona la IAT, no esconde el OEP, etc., así que poca cosa hace el pobre.

Cargamos el exe en Olly, trabajaremos sobre swishpix.exe ya que hay varios ejecutables en la carpeta de instalación, como digo en este nos centraremos que parece ser la aplicación principal, nos aparece el cartel de que el EP está fuera de la sección .code y tal, es lo más lógico cuando está empakado ya que arranca desde la sección del packer que no es la .code sino otra distinta (.pebundle en este caso es la sección) aunque hay 6 con este nombre, esto no importa para nada por eso. Aceptamos el cartel y nos pregunta si queremos analizarlo yo digo que no en estos casos, estamos en el EP:

```
00AB7000  9C            PUSHFD
00AB7001  60            PUSHAD
00AB7002  E8 02000000   CALL swishpix.00AB7009
00AB7007  33C0          XOR EAX,EAX
00AB7009  8BC4          MOV EAX,ESP
```

Vemos un PUSHFD que guarda los flags y justo después el típico PUSHAD al que seguiremos mediante el método del PUSHAD-POPAD, esto se traduce en superar la instrucción, o sea situarnos justo encima del CALL que se sigue, mirar el valor de ESP en la ventana de registros que estará en ROJO y hacer un "Follow in Dump->Memory Address" después poner un HBP en la ventana de datos (Dump) sobre la dirección que marque ESP:



Una vez tenemos puesto el HBP On Access dejamos correr el programa con F9 y para en la instrucción contraria (en un POPAD) que es este:

```
00AB73E6  61            POPAD
00AB73E7  9D            POPFD
00AB73E8  68 0050A900   PUSH swishpix.00A95000
00AB73ED  C3            RETN
```

Pero si seguimos volvemos a encontrarnos con otro PUSHAD, nada hacemos lo mismo, quitamos el HBP que teníamos puesto y le ponemos otro al siguiente PUSHAD, esta

operación se puede realizar sólo 3 veces, a partir de la cuarta vez que encontramos otro PUSHAD ya no nos sirve más el método ¿? cuando caigamos en el tercer POPAD quitamos el HBP, traceamos un poco hasta superar el RET y caemos aquí:

```
00A7C000  EB 06          JMP SHORT swishpix.00A7C008
00A7C002  68 345B2B00    PUSH 2B5B34
00A7C007  C3             RETN
00A7C008  9C             PUSHFD
00A7C009  60             PUSHAD
00A7C00A  E8 02000000    CALL swishpix.00A7C011
00A7C00F  33C0           XOR EAX,EAX
00A7C011  8BC4           MOV EAX,ESP
```

A partir de aquí cambiaremos de método, si superamos el CALL de la imagen con F8 nos aparecerá la nag en pantalla de golpe así que deberemos entrar, si nos fijamos bien apunta dos instrucciones más abajo de donde está, vamos traceando con F8 de momento llegaremos a un RET y al pasarlo entramos en una zona donde hay CALL's que desencriptan código del packer y poco después llegamos a esta zona:

```
00A7D23D  51             PUSH ECX
00A7D23E  53             PUSH EBX
00A7D23F  E8 F60B0000    CALL swishpix.00A7DE3A      Aqui dentro saca la nag.!!
00A7D244  85C0           TEST EAX,EAX                EAX==0 tras meterle mano ;-)
00A7D246  0F85 9F000000  JNZ swishpix.00A7D2EB       NO salta
00A7D24C  61             POPAD
00A7D24D  57             PUSH EDI
```

Si pasamos el CALL con F8 aparecerá la nag en pantalla así que entraremos para estudiar el CALL a ver que hace:

```
00A7DEA2  FF55 0C        CALL DWORD PTR SS:[EBP+C]
00A7DEA5  85C0           TEST EAX,EAX
00A7DEA7  74 06          JE SHORT swishpix.00A7DEAF
00A7DEA9  FFD0           CALL EAX                    swlpi.InitApp
00A7DEAB  85C0           TEST EAX,EAX                retorna AX==FFFFh
00A7DEAD  74 09          JE SHORT swishpix.00A7DEB8
00A7DEAF  B8 01000000    MOV EAX,1
00A7DEB4  C9             LEAVE
00A7DEB5  C2 1800        RETN 18
00A7DEB8  B8 00000000    MOV EAX,0
00A7DEBD  C9             LEAVE
00A7DEBE  C2 1800        RETN 18
```

En este CALL realiza una llamada a la api InitApp de la librería swlpi.dll y es donde muestra finalmente la nagscreen, podríamos seguir el CALL a ver donde nos lleva pero no lo veo necesario, superamos el call con F8 y aparece la nag:

La cerramos y volvemos al código del ejecutable swishpix.exe desde el que estamos trabajando todo el rato, si miramos los registros veremos como quedó EAX:

```
Registers (MMX)
EAX 0000FFFF
ECX 77E5A39C kernel32.77E5A39C
EDX 7FFE0304
EBX FFBECE30
ESP 0012FF60
EBP 0012FF60
ESI 00A804D4 swishpix.00A804D4
EDI 00A7E23A swishpix.00A7E23A

EIP 00A7DEAB swishpix.00A7DEAB
```

Vemos que quedó AX==FFFFh, esto le está diciendo al programa que no tenemos acceso al programa, recordad que hemos cerrado la NAG sin pulsar Try ni nada o sea que no nos va a dejar pasar, además si nos fijamos detrás del CALL hay un "TEST EAX,EAX" que mira precisamente si es EAX==0, en caso afirmativo nos lleva a :A7DEB8, se supone que deberíamos llegar allí, cambiamos el valor de EAX del que tiene por un 0, dejamos entonces EAX==0 para que salte y nos llevé para allí, si pulsamos F9 entramos en el programa así que vamos por el buen camino, aunque esto no lo vamos a hacer porque vamos a descubrir el OEP, salimos de la rutina con EAX==0:

```
00A7D23C   57              PUSH EDI
00A7D23D   51              PUSH ECX
00A7D23E   53              PUSH EBX
00A7D23F   E8 F60B0000     CALL swishpix.00A7DE3A          Aqui dentro saca la nag.!!
00A7D244   85C0            TEST EAX,EAX                    EAX==0 tras meterle mano ;-)
00A7D246   0F85 9F000000   JNZ swishpix.00A7D2EB           NO salta
00A7D24C   61              POPAD
00A7D24D   57              PUSH EDI
00A7D24E   AD              LODS DWORD PTR DS:[ESI]
00A7D24F   85C0            TEST EAX,EAX
00A7D251   0F84 9B000000   JE swishpix.00A7D2F2
00A7D257   8BD0            MOV EDX,EAX
00A7D259   0395 E6904000   ADD EDX,DWORD PTR SS:[EBP+4090E6]
```

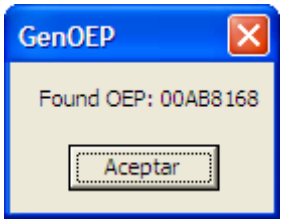
Y no va a saltar, si lo hiciera nos enviaría bastante por debajo de la llamada lo que no es muy buena señal, como le hemos cambiado el valor de retorno nos quedamos, jejeje.. a partir de aquí volvemos a tracear un poco a mano, no queda mucho vamos deshaciendo los bucles con un BP al final y pulsando F9, no olvidar ir quitando los BP's por si las moscas tuviera CRC o algo así, nunca se sabe, sólo traceando un poco con F8 enseguida llegamos al salto al OEP:

```
00A7D54E   61              POPAD
00A7D54F   9D              POPFD
00A7D550   50              PUSH EAX
00A7D551   68 345B6B00     PUSH swishpix.006B5B34          pushea la direccion del OEP
00A7D556   C2 0400         RETN 4                          y salta hasta el...
```

Ya está, jeje.. hemos llegado al OEP, cuando superamos el RET caemos sobre él:

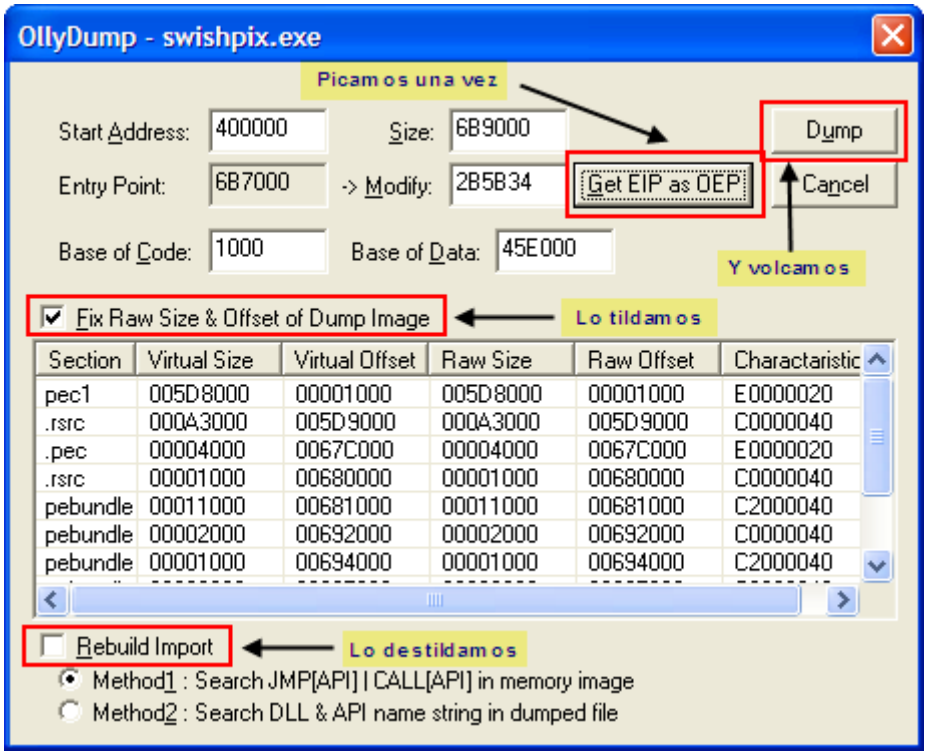
```
006B5B34   55              PUSH EBP
006B5B35   8BEC            MOV EBP,ESP
006B5B37   6A FF           PUSH -1
006B5B39   68 10B88800     PUSH swishpix.0088B810
006B5B3E   68 ECE06B00     PUSH swishpix.006BE0EC
006B5B43   64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
006B5B49   50              PUSH EAX
006B5B4A   64:8925 000000  MOV DWORD PTR FS:[0],ESP
006B5B51   83EC 58         SUB ESP,58
006B5B54   53              PUSH EBX
006B5B55   56              PUSH ESI
006B5B56   57              PUSH EDI
006B5B57   8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
006B5B5A   FF15 B0479D00   CALL DWORD PTR DS:[9D47B0]       kernel32.GetVersion
006B5B60   33D2            XOR EDX,EDX
```

Pues lo dicho, después de tanto traceo y traceo llegamos al OEP, curiosamente si le hubiéramos preguntado al PEiD no nos hubiera dado un OEP válido:



Lo dicho, la verdad que el PEiD no se suele equivocar mucho, bueno en realidad si se equivoca bastante porque no decirlo también pero tiene muy buen ojo con los packers ya que suele atinar bastante con ellos, para mi gusto es una buena herramienta (al menos en mi opinión).

Ya que estamos en el OEP podemos aprovechar para volcar el proceso, yo utilizo el plugin del Olly (Ollydump) porque me va bastante bien, se podría hacer con el del plugin IsDebuggerPresent o bien con el LordPE, en fin cualquiera vale, volcamos...



Lo que debemos hacer es destildar la opción "Rebuild Import" y tildar la opción "Fix Raw Size..." que en realidad ya viene tildada por defecto, después pulsar sobre el botón "Get EIP as OEP" para que nos deje el OEP correcto que es donde estamos ahora (el EIP actual) y por fin pulsar en "Dump" le damos el nombre dump.exe por ejemplo o el que queráis y ya tenemos un dumpeado ;-)

Ahora a por la IAT, en el código del packer y antes de saltar al OEP lógicamente se lleva a cabo el tratamiento de la IAT, justo en este loop:

```
00A7D5C4  8B19          MOV EBX,DWORD PTR DS:[ECX]          swishpix.005D545C
00A7D5C6  83C1 04       ADD ECX,4
00A7D5C9  0BDB          OR EBX,EBX
00A7D5CB  0F84 7D000000 JE swishpix.00A7D64E
00A7D5D1  8BC3          MOV EAX,EBX
00A7D5D3  F7C3 00000080 TEST EBX,80000000
00A7D5D9  74 18         JE SHORT swishpix.00A7D5F3
00A7D5DB  81E3 FFFF0000 AND EBX,0FFFF
00A7D5E1  C785 2D974000 MOV DWORD PTR SS:[EBP+40972D],1
00A7D5EB  899D 25974000 MOV DWORD PTR SS:[EBP+409725],EBX
00A7D5F1  EB 14         JMP SHORT swishpix.00A7D607
00A7D5F3  C785 2D974000 MOV DWORD PTR SS:[EBP+40972D],0
00A7D5FD  03DA          ADD EBX,EDX
00A7D5FF  43            INC EBX
00A7D600  43            INC EBX
00A7D601  899D 25974000 MOV DWORD PTR SS:[EBP+409725],EBX
00A7D607  51            PUSH ECX
00A7D608  52            PUSH EDX
00A7D609  80BD 6B974000 CMP BYTE PTR SS:[EBP+40976B],0C3
00A7D610  74 23         JE SHORT swishpix.00A7D635
00A7D612  8D8D 6BA14000 LEA ECX,DWORD PTR SS:[EBP+40A16B]
00A7D618  53            PUSH EBX
00A7D619  FFB5 57974000 PUSH DWORD PTR SS:[EBP+409757]
00A7D61F  6A 40         PUSH 40
00A7D621  51            PUSH ECX
00A7D622  FFB5 3D974000 PUSH DWORD PTR SS:[EBP+40973D]
00A7D628  FFB5 39974000 PUSH DWORD PTR SS:[EBP+409739]
00A7D62E  E8 07020000   CALL swishpix.00A7D83A
00A7D633  EB 0D         JMP SHORT swishpix.00A7D642
00A7D635  53            PUSH EBX
00A7D636  FFB5 57974000 PUSH DWORD PTR SS:[EBP+409757]
00A7D63C  FF95 3D974000 CALL DWORD PTR SS:[EBP+40973D]
00A7D642  5A            POP EDX
00A7D643  59            POP ECX
00A7D644  0BC0          OR EAX,EAX
00A7D646  74 08         JE SHORT swishpix.00A7D650
00A7D648  AB            STOS DWORD PTR ES:[EDI]             rellena la entrada.!!
00A7D649  E9 76FFFFFF   JMP swishpix.00A7D5C4
00A7D64E  F8            CLC
00A7D64F  C3            RETN
00A7D650  F9            STC
00A7D651  C3            RETN
00A7D652  C3            RETN
```

En él se va escribiendo toda la tabla, esto no es necesario tracearlo porque como ya dijimos antes no redirige ningún puntero, como curiosidad decir que nos la pone a partir de :9D4288 que está dentro de la sección .code, normalmente los ejecutables suelen tener una sección especial para albergar la iat (.rdata, .idata, etc..,) pero no es el caso, los datos de la misma los ponemos en el Import Reconstructor:

IAT Infos needed

OEP 002B5B34 IAT AutoSearch

RVA 005D4288 Size 0000108C

y pulsamos "Get Imports" nos dice que quedaron dos punteros sin resolver ¿?

Imported Functions Found

```
rva:005D47B0 mod:kernel32.dll ord:01D4 name:GetVersion
rva:005D47B4 mod:kernel32.dll ord:00EA name:FreeLibrary
rva:005
rva:005
rva:005                                  name:FindClose
rva:005                                  name:GlobalAlloc
rva:005                                  name:GlobalLock
rva:005                                  name:GetLastError
rva:005                                  name:GlobalUnlock
```

Invalidate function(s)
Disassemble / HexView
Trace Level1 (Disasm)
Trace Level2 (Hook)
Trace Level3 (Trap Flag)
Plugin Tracers

Vamos a ver hacia donde apunta cada uno de ellos:

DISASSEMBLER / HEX VIEWER

```
00A9263E    push edi
00A9263F    push ebx
00A92640    call 00A92645
00A92645    pop ebx
00A92646    sub ebx,403645
00A9264C    cmp byte ptr [ebx+40389B],1
00A92653    jnz short 00A92662
00A92655    lea eax,[ebx+4038A0]     // DWORD value: 04600BE9
00A9265B    push eax
00A9265C    call [ebx+403A42]     // DWORD value: 27E90011
00A92662    cmp byte ptr [ebx+40389A],1
00A92669    jnz short 00A9269C
00A9266B    lea eax,[ebx+403862]     // DWORD value: 37E90006
00A92671    push eax
00A92672    call [ebx+403A22]     // DWORD value: 000A3FBA
00A92678    mov edi,eax
00A9267A    mov ecx,A
00A9267F    push ecx
00A92680    push edi
00A92681    call [ebx+403A16]     // DWORD value: 2834E900
```

00A9263E Go to OpCodes Hex View OK

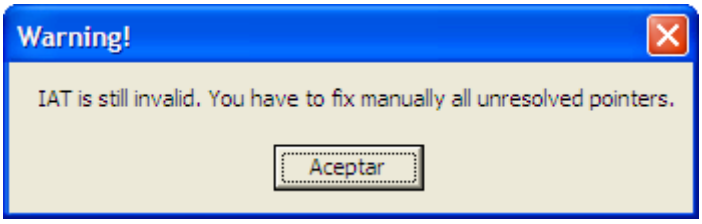
Apunta a una dirección que cae dentro del ejecutable así que no nos dará problemas, veamos la otra entrada a ver qué tal:

DISASSEMBLER / HEX VIEWER

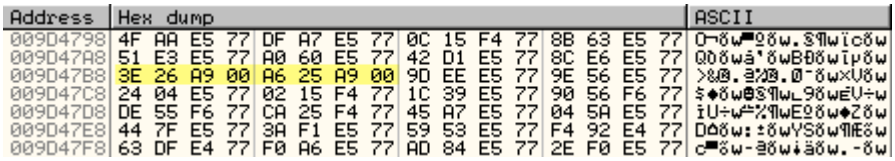
```
00A925A6    enter 0,0
00A925AA    push esi
00A925AB    push ecx
00A925AC    call 00A925B1
00A925B1    pop esi
00A925B2    sub esi,4035B1
00A925B8    mov eax,[ebp+8]
00A925BB    test eax,eax
00A925BD    je short 00A925E3
00A925BF    lea ecx,[esi+403862]     // DWORD value: 37E90006
00A925C5    push 0
00A925C7    push eax
00A925C8    push ecx
00A925C9    call 00A923EE
00A925CE    test eax,eax
00A925D0    jnz short 00A925E3
00A925D2    lea eax,[esi+403F68]     // DWORD value: 0011D854
00A925D8    push eax
00A925D9    call [esi+403A36]     // DWORD value: 0016AA66
00A925DF    test eax,eax
```

00A925A6 Go to OpCodes Hex View OK

Pues tres cuartos de lo mismo apunta dentro así que no habrá problema, haremos una cosa NO haremos "Cut Thunks" ok? No lo haremos que quede presente sino "Fix Dump" directamente, no importa que queden estas dos entradas así, el Import nos dirá esto:

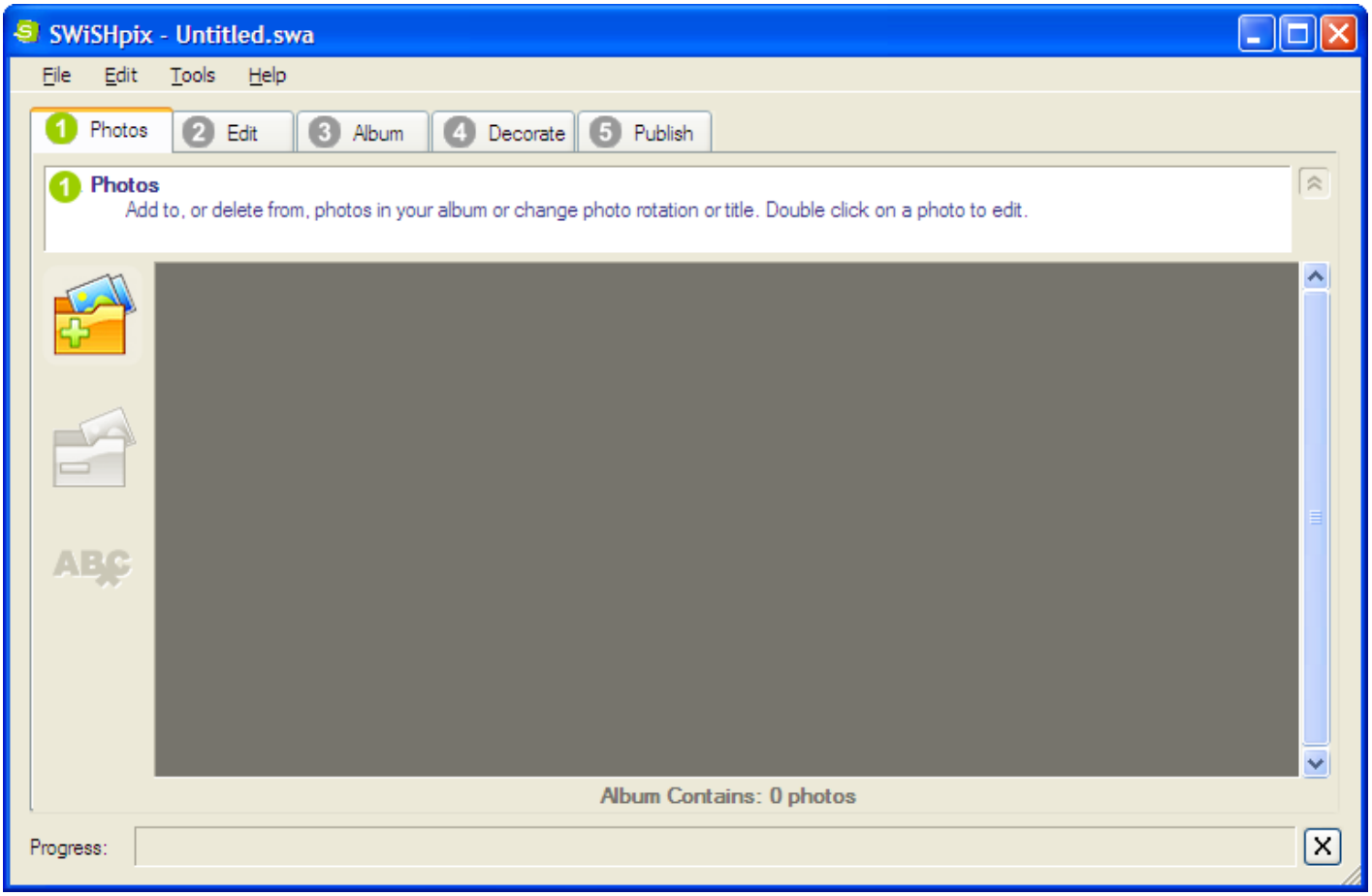


Pero repito que no importa, si abrimos el archivo dump_.exe en Olly y vamos a las direcciones donde están las entradas inválidas veremos:



Address	Hex dump	ASCII
009D4798	4F AA E5 77 DF A7 E5 77 0C 15 F4 77 8B 63 E5 77	O¬ŏw▀§ŏw.§¶wïcŏw
009D47A8	51 E3 E5 77 A0 60 E5 77 42 D1 E5 77 8C E6 E5 77	QΘŏwá'ŏwBÐŏwîµŏw
009D47B8	3E 26 A9 00 A6 25 A9 00 9D EE E5 77 9E 56 E5 77	>&@.ª%@.Ø¯ŏw×Vŏw
009D47C8	24 04 E5 77 02 15 F4 77 1C 39 E5 77 90 56 F6 77	$♦ŏw☻§¶w∟9ŏwÉV÷w
009D47D8	DE 55 F6 77 CA 25 F4 77 45 A7 E5 77 04 5A E5 77	ïU÷w╧%¶wE§ŏw♦Zŏw
009D47E8	44 7F E5 77 3A F1 E5 77 59 53 E5 77 F4 92 E4 77	D⌂ŏw:±ŏwYSŏw¶Æŏw
009D47F8	63 DF E4 77 F0 A6 E5 77 AD 84 E5 77 2E F0 E5 77	c▀ŏw-ªŏw↓äŏw.-ŏw

como vemos nos reparó la IAT perfecta, quedaron dos entradas que como apuntan a direcciones propias del ejecutable no nos deben de preocupar, si ejecutamos ahora el archivo dump_.exe veremos que funciona a la perfección:



El dumpeado ya funciona..!!

Además entramos sin la nag por lo que nos saltamos el Time-Trial de un tirón ;-) el programa queda crackeado con sólo desempakar así que podemos dar por finalizado nuestro trabajo por hoy, los objetivos han sido cubiertos.

Como curiosidad sólamente decir que el packer realiza bastante trabajo en este CALL:

```
00AB7245    FFB5 5E384000   PUSH DWORD PTR SS:[EBP+40385E]
00AB724B    E8 180F0000     CALL swishpix.00AB8168
```

después llega a:

```
00AB726B    FF95 523A4000   CALL DWORD PTR SS:[EBP+403A52]        ; kernel32.WriteFile
```

Si miramos en la ventana de registros vemos que ESI tiene un puntero al path de un archivo temopral (una dll) -> "C:\DOCUME~1\SyXe'05\CONFIG~1\Temp\\swlpi.dll" o sea que crea esta librería en tiempo de ejecución.

Al siguiente RET llegamos a:

```
00A95000    9C              PUSHFD
00A95001    60              PUSHAD
00A95002    E8 02000000     CALL swishpix.00A95009
```

Si superamos el CALL con F8 nos saca la NAG, si entramos en el CALL veremos:

```
00A951DD    FF95 0A3A4000   CALL DWORD PTR SS:[EBP+403A0A]       ;kernel32.CreateFileA
```

Miramos al stack y vemos "C:\DOCUME~1\SyXe'05\CONFIG~1\Temp\\swl.ini" que es el archivo a crear, luego lo rellena en:

```
00A9526B    FF95 523A4000   CALL DWORD PTR SS:[EBP+403A52]        ;kernel32.WriteFile
```

luego vemos:

```
00A921DD    FF95 0A3A4000   CALL DWORD PTR SS:[EBP+403A0A]       ;kernel32.CreateFileA
```

Otra vez CreateFileA, miremos al stack: "C:\DOCUME~1\SyXe'05\CONFIG~1\Temp\\image.bmp" va a crear una imagen, traceamos un poco más y enseguida llegamos a esta parte:

```
00A7D23F    E8 F60B0000     CALL swishpix.00A7DE3A   ; Aqui dentro saca la nag.!!
00A7D244    85C0            TEST EAX,EAX             ; EAX==0 tras meterle mano ;-)
00A7D246    0F85 9F000000   JNZ swishpix.00A7D2EB    ; NO salta
```

Trata la IAT en este loop:

```
00A7D56B    E8 0F000000     CALL swishpix.00A7D57F
00A7D570    72 0B           JB SHORT swishpix.00A7D57D
00A7D572    83C6 14         ADD ESI,14
00A7D575    837E 0C 00      CMP DWORD PTR DS:[ESI+C],0
00A7D579  ^ 75 F0           JNZ SHORT swishpix.00A7D56B
```

Dentro del CALL es donde realmente la va rellenando desde :9D4288 hasta :9D5314 incluídos ambos punteros y finalmente salta al OEP que está en :6B5B34.

esto sólo lo pongo como orientación por si alguien quiere estudiar más a fondo la protección, yo desde luego no tengo tiempo para ello.

Espero que os haya entretenido el tute tanto como a mi escribirlo, ya sabéis donde me podéis encontrar como siempre en:

syxe00@yahoo.es